

Experiment No. 6 - Develop a Node.js Application using Express.js and Mongoose to Connect with MongoDB

Aim: To build a robust Node.js backend application with Express.js and Mongoose ODM, establishing managed database connection pools with retry logic, strongly-typed Schemas with field validations, pre-save middleware hooks, and cross-collection relational population.

Tools Used:

- Node.js Runtime Environment (v22.x)
- Express.js (Web Framework)
- Mongoose (Object Data Modeling Library)
- dotenv (Environment Configuration Management)
- Visual Studio Code

Theory:

While MongoDB provides flexible document storage, enterprise production backends require strict schema integrity, type enforcement, and automated business logic hooks. Mongoose serves as the premier Object Data Modeling (ODM) library for Node.js, bridging the gap between JavaScript application code and MongoDB's BSON store.

Mongoose establishes connection pooling via `mongoose.connect()`, listening to connection lifecycle events (connected, error, disconnected) to maintain database availability. Models are constructed from Schemas, defining data types (String, Number, Date, ObjectId), required constraints, default values, and custom validators.

Mongoose middleware (pre and post hooks) enables automated lifecycle operations. For instance, in the Pet Care Management System, a `pre('save')` hook intercepts user registrations to cryptographically hash passwords using `bcryptjs` before persisting them to the database. Relational associations between disparate collections—such as linking a Pet document to its owning User—are achieved using `Schema.Types.ObjectId` references paired with `populate()`.

Functions and Methods Used: `mongoose.connect()`, `mongoose.connection.on()`, `new mongoose.Schema()`, `Schema.pre('save')`, `mongoose.model()`, and `Query.prototype.populate()`.

Methodology:

The application structure adopts an enterprise Model-Controller architecture. First, database connectivity is abstracted into `config/db.js`, reading MongoDB connection URIs securely from `.env` via `dotenv` and handling connection retries.

Second, domain schemas are defined in `models/User.js`, `models/Pet.js`, and `models/Appointment.js`. Strongly-typed field validators and pre-save password hashing hooks are embedded within the

schemas. Third, Express routes instantiate Mongoose model methods to verify schema validation and relational query resolution.

Procedure:

1. Initialize an Express project with `npm init -y` and install dependencies: `express`, `mongoose`, `dotenv`, `bcryptjs`.
2. Create `.env` file specifying `PORT=5000` and `MONGO_URI=mongodb://localhost:27017/petcare_db`.
3. Implement `config/db.js` using `mongoose.connect()` with event listeners for `connected` and `error`.
4. Define `models/User.js` schema specifying `name`, `email` (unique index), `password`, and `role` with `enum` restrictions.
5. Attach a Mongoose pre-save hook in `User.js` to hash plain-text passwords using `bcrypt`.
6. Define `models/Pet.js` schema specifying pet attributes and referencing the owner via `type ObjectId`.
7. Create `server.js` mounting Express JSON parsers and invoking the database connection script.
8. Start the application using `node server.js` and verify database connection messages in the terminal.
9. Test Mongoose validation error handling by attempting to save a pet document missing mandatory fields.
10. Execute a populated query `Pet.find().populate('owner', 'name email phone')` to verify cross-collection references.

Code :

// File: backend/config/db.js & models/Pet.js

```
// backend/config/db.js - Database Connection
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI ||
'mongodb://localhost:27017/petcare_db');
    console.log(`MongoDB Connected Successfully: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Database Connection Error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;

// backend/models/Pet.js - Mongoose Schema with Relationships
const petSchema = new mongoose.Schema({
  name: { type: String, required: [true, 'Pet name is required'], trim: true },
  species: { type: String, required: true, enum: ['Dog', 'Cat', 'Bird',
'Rabbit', 'Other'] },
  breed: { type: String, required: true },
  age: { type: Number, required: true, min: [0, 'Age cannot be negative'] },
  weight: { type: Number, required: true },
  owner: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  vaccinationStatus: { type: String, enum: ['Vaccinated', 'Pending', 'Overdue'],
default: 'Pending' },
}, { timestamps: true });

module.exports = mongoose.model('Pet', petSchema);
```

Output:

Console: Terminal - Node.js Server Startup

```
PS C:\Users\tanuj\pet-care-management-system\backend> node server.js
[dotenv] Loaded environment variables from .env
MongoDB Connected Successfully: 127.0.0.1
Pet Care API Server running in development mode on http://localhost:5000
Ready to accept REST API requisitions...
```

Figure 6.1 Terminal Output Confirming Successful Mongoose Connection

This screenshot displays the server console startup logs. It verifies that dotenv loaded the environment variables and Mongoose established a healthy connection to the database.

Console: VS Code - .env Configuration

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/petcare_db
JWT_SECRET=super_secret_petcare_jwt_key_2026
NODE_ENV=development
CLIENT_URL=http://localhost:5173
```

Figure 6.2 Environment Variable Configuration (.env) Loaded via dotenv

This screenshot shows the isolated .env configuration file containing the connection string and port settings, preventing hardcoded credentials in application source code.

POSTMAN | POST http://localhost:5000/api/pets

Status: 400 Bad Request | 14 ms

```
{
  "success": false,
  "error": "Validation Error: Pet name is required, species is required"
}
```

Figure 6.3 Mongoose Schema Validation Intercepting Missing Pet Name

This screenshot illustrates Mongoose's built-in schema validation. Attempting to insert a pet document without the mandatory name field produces a 400 Bad Request error.

Console: Node.js Console - Pre-Save Hook Execution

```
[Mongoose Hook] pre('save') triggered for user: tanuj.sharma@petcare.org
[Bcrypt] Generating 10 salt rounds...
[Bcrypt] Plaintext password transformed to: $2a$10$e8T7rK91bQzJ9L1X...
Document committed to 'users' collection with secure cryptographic hash.
```

Figure 6.4 Mongoose Pre-Save Hook Executing Bcrypt Password Hashing

This figure captures the pre-save hook executing before user document persistence. The plain-text password is encrypted with 10 salt rounds before saving to the database.

```
POSTMAN | GET http://localhost:5000/api/pets/bruno-id Status: 200 OK | 32 ms

{
  "_id": "6701844af19c927d3b018401",
  "name": "Bruno",
  "species": "Dog",
  "breed": "Labrador",
  "owner": {
    "_id": "67018300f19c927d3b018390",
    "name": "Tanuj Sharma",
    "email": "owner@petcare.com",
    "phone": "9876543210"
  },
  "vaccinationStatus": "Vaccinated"
}
```

Figure 6.5 Mongoose Population Resolving Pet Owner Reference

This screenshot displays the output of `Pet.find().populate('owner')`. Mongoose resolves the owner `ObjectId` reference into a full user profile containing name, email, and phone.

```
Console: Terminal - Connection Resiliency Test

[Mongoose] Warning: Lost MongoDB connection. Attempting auto-reconnect...
[Mongoose] Retrying connection attempt 1 of 5...
Mongoose successfully reconnected to petcare_db cluster.
```

Figure 6.6 Database Connection Retry and Graceful Reconnection

This screenshot shows Mongoose's resilient reconnection behavior. When the database daemon undergoes a transient restart, Mongoose logs a warning and automatically reconnects.

```
MongoDB Compass - [Collection: pets]

Document: 6701844af19c927d3b018401
_id: ObjectId("6701844af19c927d3b018401")
name: "Bruno"
owner: ObjectId("67018300f19c927d3b018390")
vaccinationStatus: "Vaccinated"
__v: 0 // Mongoose Concurrency Version Key
```

Figure 6.7 MongoDB Compass Viewing Mongoose Version Key(__v)

This screenshot shows documents inside MongoDB Compass. It highlights Mongoose's internal version key (`__v: 0`) and automatic timestamps (`createdAt`, `updatedAt`).

```
Console: Backend Directory Layout

backend/
├── config/
│   └── db.js          <-- Mongoose connection pool
├── controllers/
│   ├── authController.js
│   ├── petController.js
│   └── appointmentController.js
├── models/
│   ├── User.js        <-- User Schema & hooks
│   ├── Pet.js         <-- Pet Schema with ref
│   └── Appointment.js  <-- Appointment Schema
├── routes/
│   └── petRoutes.js
├── .env
└── server.js
```

Figure 6.8 Project Structure Showing Modular Backend Architecture

This screenshot captures the clean, modular backend architecture in VS Code, showing structured separation across config, controllers, models, and routes.

Conclusion:

Experiment No. 6 successfully established a production-grade backend data architecture using Express.js and Mongoose ODM to connect with MongoDB. Implementing connection abstraction with event listeners provided fault tolerance and automated reconnection capabilities.

Designing strongly typed Mongoose Schemas introduced strict validation rules directly at the data model level, preventing invalid or malformed data from persisting to the database. The implementation of pre-save hooks handled sensitive security workflows, specifically encrypting passwords with bcrypt prior to database write operations.

Finally, utilizing ObjectId references and Mongoose's populate() method enabled seamless relational querying between pets, owners, and appointments within a NoSQL document database. This established a robust foundation for building RESTful APIs.